Scalability in System Design

1. Introduction

When designing systems, we **start simple** (e.g., single server) and gradually evolve the architecture as traffic grows.

The goal is:

> To ensure the system can **handle increasing (and decreasing) load efficiently**

2. Definition

Scalability is the ability of a system to:

> **efficiently and cost-effectively handle changes in load by adjusting resources (up or down)**

✔ Handles:

- More users
- More requests
- More data

✔ Also supports:

- Scaling **down** (cost optimization)

3. Why Scalability is Important

Without scalability:

- System crashes under load
- High latency
- Poor user experience

- Revenue loss

Real-world example:

- During sales (Flipkart/Amazon), traffic spikes → system must scale

4. Types of Scaling

Vertical Scaling (Scale Up)

Increase resources in a single machine:

- CPU
- RAM
- Storage

Pros:

- Simple
- No architecture change

Cons:

- Limited capacity
- Expensive
- Single point of failure

Horizontal Scaling (Scale Out)

Add more machines to handle load

Pros:

- Highly scalable
- Fault tolerant
- Industry standard

Cons:

- Complex

- Requires distributed systems design

5. Dimensions of Scalability (Important Addition)

1. Compute Scalability

- Adding more servers

2. Storage Scalability

- Handling increasing data (sharding, partitioning)

3. Network Scalability

- Handling more requests per second

6. How Systems Scale

Step 1:

Single Server

Step 2:

Separate Web + Database

Step 3:

Add Load Balancer

Step 4:

Add Caching

Step 5:

Database Scaling (Replication/Sharding)

Step 6:

Microservices + Distributed Systems

7. Scale Up vs Scale Down

Scale Up (Auto-scaling ↑)

- During peak traffic

Scale Down (Auto-scaling ↓)

- During low usage → saves cost

This is critical in **cloud systems (AWS, GCP)**

8. Key Techniques for Scalability

- Load Balancing
- Caching (Redis, CDN)
- Database Replication
- Sharding
- Asynchronous Processing (Queues)
- Microservices Architecture

9. Challenges in Scalability

- Data consistency
- Distributed failures
- Network latency
- Synchronization issues
- Debugging complexity

10. Final Summary

- Scalability = ability to handle changing load
- Includes both **scale up and scale down**
- Two types: **Vertical & Horizontal**
- Horizontal scaling is preferred in large systems
- Achieved using LB, caching, DB scaling